CS 422 Assignment 4: Maximum Concurrent Flow and Network Design

CS 422 project team

November 2026

1 Submission and code map

Repository: Assignment 4 folder on GitHub.

- Part 1 topology (generalized de Bruijn): `topologies.py`, lines 10–36.
- Exact concurrent-flow LP: `concurrent_flow.py`, lines 24–69.
- Upper bound $1/h^*(n,d)$: `concurrent_flow.py`, lines 86–95.
- Part 1 evaluation and figures (one-shot: `python run.py`): `run.py`, lines 49–143.

2 Model

There are n nodes and no other switches. Each node has d outgoing and d incoming directed links of capacity 1. Nodes forward traffic for each other, and a demand may be split over several paths. Given a traffic matrix T, the concurrent flow λ is the largest value such that λT_{ij} can be routed from i to j for *every* pair at the same time without exceeding any link capacity.

3 Part 1: uniform traffic

3.1 Upper bound: what any topology can achieve

Under the uniform matrix $T_{ij} = d/(n-1)$, every node sends d in total, so the total demand is nd, which equals the total link capacity nd.

Each hop costs capacity. A unit of flow that travels h hops occupies one unit of capacity on each of h links. Routing λT_{ij} from i to j therefore uses at least $\lambda T_{ij}\,\mathrm{dist}(i,j)$ capacity. Summing over all pairs:

$$\lambda \frac{d}{n-1} \sum_{i \neq j} \mathrm{dist}(i,j) \;\leq\; nd \quad\Longrightarrow\quad \lambda \;\leq\; \frac{n(n-1)}{\sum_{i \neq j} \mathrm{dist}(i,j)} \;=\; \frac{1}{\bar{h}(G)},$$

where $\bar{h}(G)$ is the average shortest-path hop count. For uniform traffic, maximizing λ therefore means minimizing average hop count. The bound is reached exactly only if (a) all flow follows shortest paths and (b) every link is fully loaded, so load must be balanced as well.

No topology has fewer hops than a full d-ary tree. From any node, at most d nodes are 1 hop away, at most d^2 are 2 hops away, and at most d^h are h hops away (the breadth-first argument in the

topologies lecture). Filling these layers greedily, $m_h = \min\big(d^h,\ (n-1) - \sum_{k<h} m_k\big)$, gives the smallest possible average hop count $h^*(n,d) = \frac{1}{n-1}\sum_h h\, m_h$, so

$$\lambda \ \le\ 1/\bar{h}(G) \ \le\ 1/h^*(n,d) \qquad \text{for every topology } G.$$

For example, $n = 8, d = 4$ gives layers $(4, 3)$, $h^* = 10/7$, and $\lambda \le 0.7$. Directed graphs that meet this bound exactly do not exist for $d \ge 2$ and diameter ≥ 2 (Plesník and Znám, 1974), so the goal is a construction whose hop counts come close to it.

3.2 Construction: generalized de Bruijn graph

We use the de Bruijn graph from the topologies lecture ("shift left, then append one new digit"). Writing it as arithmetic makes it work for any n:

```
for u in 0 .. n-1:
    for k in 0 .. d-1:
        add link  u -> (d*u + k) mod n
```

Shifting a base-d label left is multiplying by d, and appending digit k is adding k. When $n = d^m$ this is exactly the lecture's de Bruijn graph. Working mod n is needed because the assignment's assumption $(d \mid n-1)$ rules out $n = d^m$. Each node has d outgoing links by construction, and also d incoming links: as u ranges over $0..n-1$ and k over $0..d-1$, the dn values $du+k$ hit every residue mod n exactly d times. Figure 1 shows $n = 9, d = 2$.

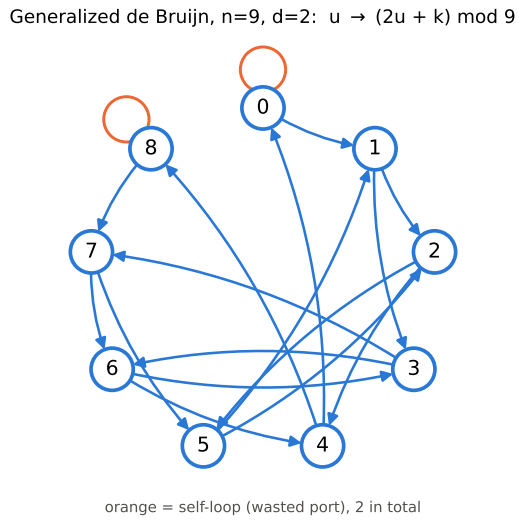


Figure 1: Generalized de Bruijn graph, $n = 9$, $d = 2$. Orange rings are self-loops.

3.3 Why this topology: short paths

Every node reaches every other in at most $\lceil \log_d n \rceil$ hops. After m hops from u with digits $k_1, \ldots, k_m$ we are at

$$d^m u + (k_1 d^{m-1} + k_2 d^{m-2} + \cdots + k_m) \pmod{n}.$$

The bracketed term can be any integer in $0, \ldots, d^m - 1$, the base-d numbers with m digits. Once $d^m \ge n$, these cover every residue mod n, so every target is reachable. This matches the breadth-first lower bound,

which says no topology can reach all n nodes in fewer than about $\log_d n$ hops. So de Bruijn's hop counts, and therefore its bound $1/\bar{h}$, are close to the best possible. For comparison, the obvious "ring with chords" (simple circulant $u \to u+1, \ldots, u+d$) reaches only d new nodes per hop, so its average hop count grows linearly in n.

3.4 The gap to the bound

As the lecture notes, "regular labels do not guarantee balanced traffic". De Bruijn misses condition (b) in two ways:

- **Self-loops.** $du + k \equiv u$ can hold. It always holds at $u = 0$ ($k = 0$) and at $u = n-1$ ($k = d-1$), and those ports carry nothing.
- **Uneven load.** The graph is not symmetric, so shortest-path routing loads some links more than others. The LP compensates by detouring flow, which costs extra hops.

So we present de Bruijn as *near-optimal with a quantified gap*, not as exactly optimal.

3.5 Evaluation

For each (n, d) with $d \mid n-1$ we computed the exact λ with a Gurobi LP (one commodity per source, arc flows with conservation and capacity constraints) and compared it with the bound $1/h^*(n,d)$ and with the simple circulant baseline (Figure 2, Table 1).

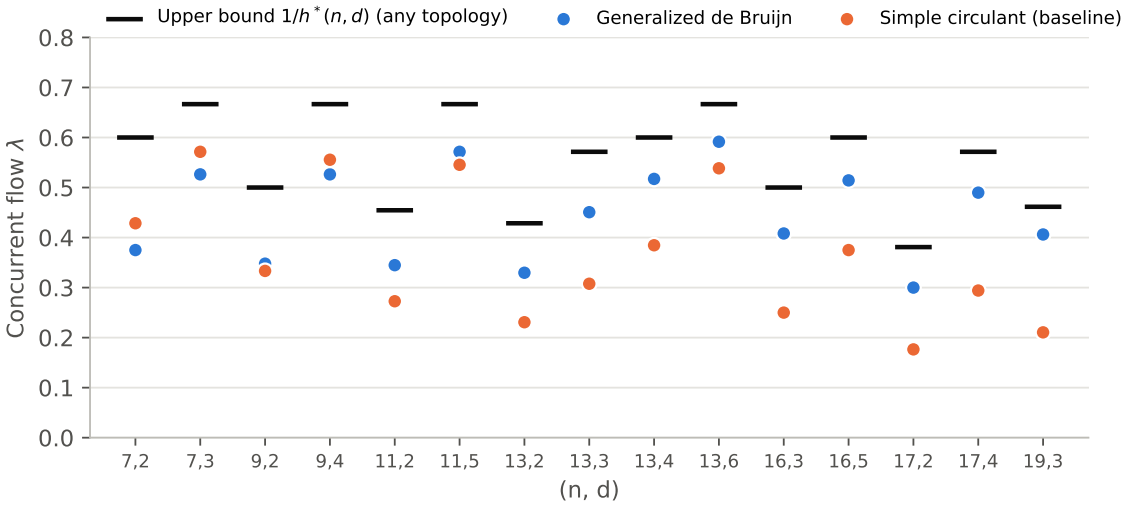


Figure 2: Exact concurrent flow under uniform traffic versus the upper bound that applies to every topology.

Findings.

- De Bruijn reaches 63–89% of the bound, which applies to every topology, and 79% or more in 11 of 15 cases (the exceptions are $d = 2$ with $n \leq 13$).
- Its maximum hop count equals $\lceil \log_d n \rceil$ in every case, confirming the argument above.
- It beats the simple circulant in 12 of 15 cases, and the margin grows with n (at $n = 19, d = 3$: 0.406 vs. 0.211). This is the logarithmic vs. linear hop-count difference.

n	d	bound $1/h^*$	de Bruijn λ	% of bound	max hops	$\lceil \log_d n \rceil$	circulant λ
7	2	0.600	0.375	63	3	3	0.429
7	3	0.667	0.526	79	2	2	0.571
9	2	0.500	0.348	70	4	4	0.333
9	4	0.667	0.526	79	2	2	0.556
11	2	0.455	0.345	76	4	4	0.273
11	5	0.667	0.571	86	2	2	0.545
13	2	0.429	0.330	77	4	4	0.231
13	3	0.571	0.451	79	3	3	0.308
13	4	0.600	0.517	86	2	2	0.385
13	6	0.667	0.592	89	2	2	0.538
16	3	0.500	0.408	82	3	3	0.250
16	5	0.600	0.514	86	2	2	0.375
17	2	0.381	0.300	79	5	5	0.176
17	4	0.571	0.490	86	3	3	0.294
19	3	0.462	0.406	88	3	3	0.211

Table 1: Part 1 results (`part1/results/part1.csv`).

- The circulant wins only for the smallest, densest cases (7/2, 7/3, 9/4). There, both graphs have nearly the same hop counts, and de Bruijn's two or more wasted self-loop ports out of only nd total cost more than its shorter paths save.

4 Part 2: arbitrary hose traffic ($n = 8$, $d = 4$)

To be written.

5 AI use

To be written (see `AI_USAGE.md`).